

Case Study: SupportSense

Designing an ML-Powered Customer Support Ticket Management System

The Scenario

A large organization receives **thousands of customer-support tickets every day** through email, web forms, mobile applications, and chat. Support agents currently read each ticket, determine its category and priority, and route it to the appropriate team.

The organization wants to build **SupportSense**, an ML-powered system that assists support agents. For each incoming ticket, the system should **classify the ticket, estimate its priority, provide a confidence score, and route it to the appropriate team**. Low-confidence cases should be sent to a human agent for review.

For example:

Ticket: “My payment was deducted but my order was not confirmed.”
Prediction: Payment Issue **Confidence:** 94% **Priority:** High

The support agent may accept or correct the prediction. These corrections provide labelled data that could potentially be used to improve future versions of the model.

The MLOps Challenge

The organization initially trains the model using historical tickets. However, after several months:

- a new payment method and new products are introduced;
- customers begin using new terminology;
- new support categories appear;
- agents correct an increasing number of predictions;
- the volume of incoming tickets increases significantly.

The organization therefore needs more than a model that is trained once. It needs an ML system that can continuously:

Operate → Monitor → Learn → Retrain → Validate → Redeploy

Questions to Think About

You are part of the team designing the MLOps solution. Come prepared to discuss the following:

1. **Data:** How should incoming tickets be collected and processed? What happens when thousands arrive simultaneously? How can tickets be prevented from being lost?
2. **Model:** What ML model would you choose for ticket classification? What features would you use? How would you generate a confidence score?
3. **Decision:** Should every prediction be automatically used? What should happen when the model is uncertain?
4. **Serving:** How should the model be deployed? What are the requirements for latency, CPU/GPU, memory, scalability, and cost?
5. **Monitoring:** What should be monitored at three levels: *infrastructure*, *ML model*, and *application usage*?
6. **Feedback:** How should corrections made by support agents be stored? Should every correction immediately become training data?
7. **Retraining:** When should the model learn again? Consider scheduled retraining, feedback volume, data drift, performance degradation, and introduction of new categories.
8. **Validation:** A new model has been trained. Should it immediately replace the production model? What conditions should it satisfy before deployment?
9. **Reliability:** What should happen if the ML service fails while tickets continue to arrive?
10. **Scaling:** When would a simple deployment no longer be sufficient? How might the architecture evolve as ticket volume grows?
11. **Evolution:** If the system eventually serves multiple organizations, what new requirements would appear?

Your Task

Design and justify an end-to-end MLOps solution:

Data → Training → Deployment → Monitoring → Feedback → Retraining → Validation → Redeployment

There is **no single correct architecture**. Be prepared to explain **why** you made each design decision.

Think beyond model accuracy.

A production ML system must not only make good predictions. It must remain **reliable, observable, useful, scalable, and capable of improving as the data and business environment change.**